

LABSHEET -5

SUBMITTED BY : UTSAV KUMAR KUSHWAHA
 ROLL NO : 63
 CUID : CU240251668
 SEC : "B"

SUBMITTED TO : AKSHAY JUNEJA

Feature Extraction and Image Analysis using SIFT and HOG Descriptors

Aim

To implement Scale-Invariant Feature Transform (SIFT) and Histogram of Oriented Gradients (HOG) techniques for feature extraction and analyze their effectiveness in image representation, object recognition, and computer vision applications.

Purpose

Feature extraction is one of the most important stages in computer vision because it converts raw pixel information into meaningful numerical representations called feature descriptors. Instead of processing every pixel, algorithms focus on important structures such as edges, corners, and textures.

SIFT and HOG are two widely used handcrafted feature extraction techniques.

- SIFT identifies distinctive keypoints that remain stable under changes in scale and rotation.
- HOG captures gradient orientation patterns, making it highly effective for object detection.

These descriptors are used in image matching, object recognition, face recognition, tracking, and many other vision applications.

Theory

What is Feature Extraction?

Feature extraction is the process of identifying important characteristics of an image and representing them as numerical vectors.

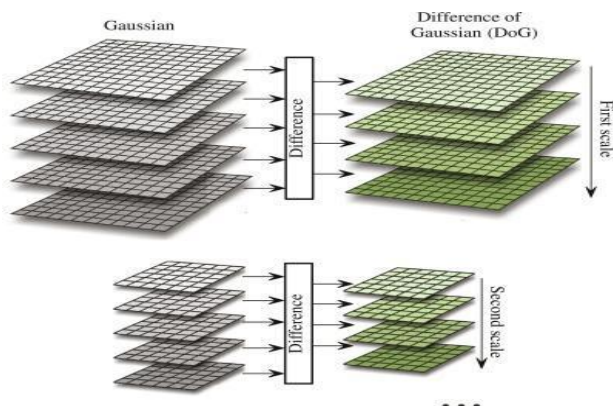
Instead of using all pixels, the algorithm extracts information like:

- Corners
- Edges
- Texture
- Gradient patterns
- Interest points

Benefits

- Reduces data size
- Improves processing speed
- Increases recognition accuracy
- Makes algorithms more robust

Scale-Invariant Feature Transform (SIFT)



SIFT is a feature detection and description algorithm developed by David Lowe (2004). It detects highly distinctive keypoints and creates descriptors that remain stable under:

- Scale changes
- Rotation
- Moderate illumination changes
- Small viewpoint variations

Working Steps of SIFT

1. Scale Space Construction – Gaussian blur is applied at multiple scales.
2. Difference of Gaussian (DoG) – Neighboring blurred images are subtracted.
3. Keypoint Detection – Local maxima and minima are selected.
4. Orientation Assignment – Each keypoint receives a dominant orientation.
5. Descriptor Generation – A 128-dimensional descriptor is created.

SIFT Pipeline

Histogram of Oriented Gradients (HOG)

HOG is a feature descriptor introduced by Dalal and Triggs (2005).

It represents an image using the distribution of edge directions instead of individual pixels.

Working Steps

1. Convert image to grayscale.
2. Calculate horizontal and vertical gradients.
3. Divide image into small cells.
4. Build gradient orientation histograms.
5. Normalize neighboring blocks.
6. Concatenate all histograms into one feature vector.

HOG Process

Prerequisites

- Python
- OpenCV
- NumPy
- Matplotlib
- scikit-image
- Jupyter Notebook or VS Code

Required Libraries

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage.feature import hog
from skimage import exposure
```

Python Program

Step 1: Load Image

```

from google.colab import files
import cv2
import matplotlib.pyplot as plt
import numpy as np

# Image upload
uploaded = files.upload()

# Automatically uploaded file ka path le lo
image_path = list(uploaded.keys())[0]

# Image read
image = cv2.imread(image_path)

# Check image loaded or not
if image is None:
    print("Image load nahi hui.")
else:
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    plt.figure(figsize=(4,3))
    plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
    plt.title("Original Image")
    plt.axis("off")
    plt.show()

```

Snapchat-342047767.jpg

Snapchat-342047767.jpg (image/jpeg) - 425619 bytes, last modified: 1/16/2025 - 100% done
 Saving Snapchat-342047767.jpg to Snapchat-342047767 (4).jpg

Original Image



Step 2: SIFT Feature Detection

```

sift = cv2.SIFT_create()

keypoints, descriptors = sift.detectAndCompute(gray, None)

sift_image = cv2.drawKeypoints(
    image,
    keypoints,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

plt.imshow(cv2.cvtColor(sift_image, cv2.COLOR_BGR2RGB))
plt.title("SIFT Keypoints")

```

```
plt.axis("off")
plt.show()
```

SIFT Keypoints



Step 3: HOG Feature Extraction

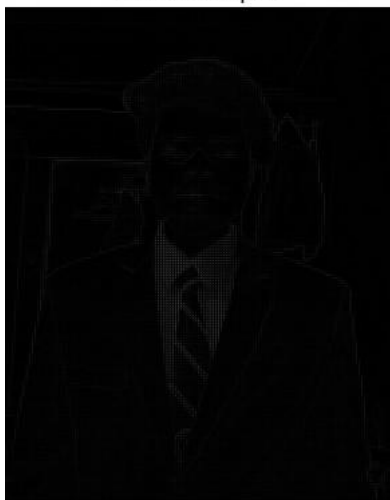
```
from skimage.feature import hog
from skimage import exposure

hog_features, hog_image = hog(
    gray,
    orientations=9,
    pixels_per_cell=(8,8),
    cells_per_block=(2,2),
    visualize=True
)

hog_image = exposure.rescale_intensity(hog_image)

plt.imshow(hog_image, cmap="gray")
plt.title("HOG Descriptor")
plt.axis("off")
plt.show()
```

HOG Descriptor



Step 4: Image Matching using SIFT

```
from google.colab import files

# Second image upload
uploaded2 = files.upload()
```

```

# Automatically second image ka path le lo
image2_path = list(uploaded2.keys())[0]

# Read second image
image2 = cv2.imread(image2_path)

# Convert to grayscale
gray2 = cv2.cvtColor(image2, cv2.COLOR_BGR2GRAY)

# Detect keypoints and descriptors
kp2, des2 = sift.detectAndCompute(gray2, None)

# Brute Force Matcher
bf = cv2.BFMatcher()

matches = bf.knnMatch(descriptors, des2, k=2)

# Lowe's Ratio Test
good = []
for m, n in matches:
    if m.distance < 0.75 * n.distance:
        good.append(m)

# Draw Matches
matched = cv2.drawMatches(
    image,
    keypoints,
    image2,
    kp2,
    good,
    None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)

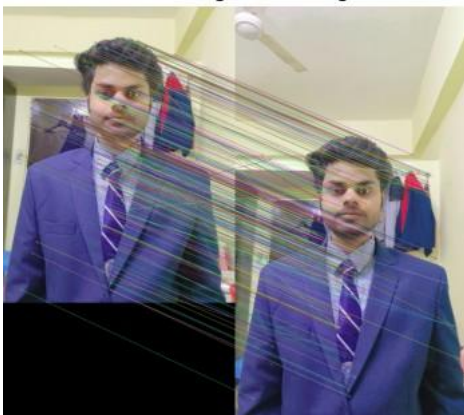
# Show Result
plt.figure(figsize=(5,4))
plt.imshow(cv2.cvtColor(matched, cv2.COLOR_BGR2RGB))
plt.title("SIFT Image Matching")
plt.axis("off")
plt.show()

print("Good Matches:", len(good))

```

Choose Files Snapchat-490056817.jpg
Snapchat-490056817.jpg (image/jpeg) - 1357955 bytes, last modified: 1/16/2025 - 100% done
 Saving Snapchat-490056817.jpg to Snapchat-490056817.jpg

SIFT Image Matching



Good Matches: 424

Step 5: Extract HOG Features

```

from skimage.feature import hog
from skimage import exposure

hog_features, hog_image = hog(
    gray,
    orientations=9,

```

```

pixels_per_cell=(8,8),
cells_per_block=(2,2),
visualize=True
)

print("HOG Feature Vector Length:", len(hog_features))

```

HOG Feature Vector Length: 824904

Step 6: Visualize HOG Descriptor

```

hog_image = exposure.rescale_intensity(hog_image)

plt.figure(figsize=(4,3))
plt.imshow(hog_image, cmap="gray")
plt.title("HOG Descriptor")
plt.axis("off")
plt.show()

```

HOG Descriptor



Step 7: Compare SIFT and HOG

```

print("SIFT Keypoints:", len(keypoints))
print("HOG Feature Length:", len(hog_features))

```

SIFT Keypoints: 1920
HOG Feature Length: 824904

Step 8: Image Matching using SIFT

```

uploaded2 = files.upload()
image2_path = list(uploaded2.keys())[0]

image2 = cv2.imread(image2_path)
gray2 = cv2.cvtColor(image2, cv2.COLOR_BGR2GRAY)

```

Snapchat-1...6761055.jpg
Snapchat-1186761055.jpg (image/jpeg) - 189525 bytes, last modified: 1/16/2025 - 100% done
 Saving Snapchat-1186761055.jpg to Snapchat-1186761055 (1).jpg

Matching Code

```

kp2, des2 = sift.detectAndCompute(gray2, None)

bf = cv2.BFMatcher()

matches = bf.knnMatch(descriptors, des2, k=2)

good = []

for m, n in matches:
    if m.distance < 0.75 * n.distance:
        good.append(m)

matched = cv2.drawMatches(
    image,
    keypoints,
    image2,

```

```
kp2,
good,
None,
flags=2
)

plt.figure(figsize=(4,3))
plt.imshow(cv2.cvtColor(matched, cv2.COLOR_BGR2RGB))
plt.title("SIFT Image Matching")
plt.axis("off")
plt.show()

print("Good Matches:", len(good))
```

SIFT Image Matching



Good Matches: 63

Step 9: Observations

Lab file me ye observations likh dena.

- SIFT ne image ke corners aur textured regions me keypoints detect kiye.
- HOG ne edge aur gradient direction ko effectively represent kiya.
- SIFT image matching me better perform karta hai.
- HOG object detection ke liye faster feature representation deta hai.
- SIFT scale aur rotation changes ke baad bhi stable rehta hai.

Step 10: Result

Result:

The experiment was successfully performed using SIFT and HOG feature extraction techniques. SIFT generated robust keypoints and descriptors suitable for image matching, while HOG effectively represented gradient orientation patterns useful for object detection. The comparison showed that SIFT provides better scale and rotation invariance, whereas HOG offers faster computation for detection-based applications.